

ShoppingScanner - Dokumentation

Inhaltsverzeichnis

1. [Projektübersicht](#)
2. [Technologie-Stack](#)
3. [Projektstruktur](#)
4. [Kernfunktionen](#)
5. [Services](#)
6. [Komponenten](#)
7. [Screens](#)
8. [Datenmodelle](#)
9. [Konfiguration](#)
10. [Installation und Setup](#)

Projektübersicht

ShoppingScanner ist eine mobile Anwendung, die entwickelt wurde, um den Einkaufsprozess zu optimieren und die Verwaltung von Einkaufslisten, Budgets und Ernährungsinformationen zu vereinfachen. Die App bietet verschiedene Funktionen wie Produktscanning, Einkaufslisten-Management, Budgetverwaltung und Ernährungstracking.

Hauptfunktionen:

- Einkaufslisten-Verwaltung
- Produktscanning und -erkennung
- Kategorie-Management
- Budget-Tracking
- Ernährungsinformationen
- Statistik-Export
- Rezeptvorschläge

Technologie-Stack

- **Framework:** React Native mit Expo
- **Sprache:** TypeScript
- **State Management:** Lokaler State mit React Hooks
- **Datenpersistenz:**
 - AsyncStorage (lokal)
 - Firebase (Cloud)
- **Backend & Datenbank:** Firebase
 - Cloud Firestore
 - Authentication
 - Cloud Storage
 - Cloud Functions
- **UI-Komponenten:** Native Base & Custom Components
- **Icons:** MaterialCommunityIcons
- **APIs:**
 - Expo Camera
 - Expo FileSystem
 - Expo MediaLibrary
 - Expo Sharing
 - Firebase API

Projektstruktur

```
src/
├── components/      # Wiederverwendbare UI-Komponenten
├── screens/         # Hauptbildschirme der App
├── services/        # Geschäftslogik und Datenverarbeitung
├── types/           # TypeScript Definitionen
├── hooks/           # Custom React Hooks
├── navigation/      # Navigationskonfiguration
└── utils/           # Hilfsfunktionen
```

Kernfunktionen

1. Einkaufslisten-Management

- Erstellen und Verwalten von Einkaufslisten
- Hinzufügen/Entfernen von Produkten
- Kategorisierung von Produkten
- Mengenangaben und Einheiten
- Abhak-Funktion für gekaufte Artikel

2. Produktscanning

- Barcode-Scanning
- Automatische Produkterkennung
- Manuelle Produkteingabe

- Produkthistorie

3. Kategorie-System

- Benutzerdefinierte Kategorien
- Hierarchische Kategoriestructur
- Farbcodierung
- Icon-Auswahl
- Import/Export von Kategorien
- Mehrsprachige Kategorie-Templates
- Intelligente Kategorievorschläge
- Erweiterte Verwaltungsoptionen

4. Budget-Verwaltung

- Kategorie-basierte Budgets
- Ausgabenverfolgung
- Budget-Warnungen
- Periodenbasierte Auswertung (täglich/wöchentlich/monatlich)
- Push-Benachrichtigungen
- KI-basierte Prognosen
- Automatische Anpassungen
- Intelligente Budgetvorschläge
- Preis-Tracking und Alarne

5. Ernährungstracking

- Nährwertinformationen
- Tägliche Übersichten
- Zielvorgaben
- Trendanalysen
- Persönliche Zieldefinition
- KI-basierte Ernährungsanalyse
- Saisonale Ernährungstipps
- Fortschrittsverfolgung
- Personalisierte Empfehlungen

6. Export- und Backup-System

- Flexible Datenexporte in mehreren Formaten
 - JSON für strukturierte Daten
 - CSV für einfache Tabellenexporte
 - XML für Systemintegration
 - Excel (XLSX) für erweiterte Tabellenkalkulationen
 - PDF für formatierte Berichte
- Cloud-Integration (Implementiert)
 - Nahtlose Integration mit Google Drive
 - Dropbox-Unterstützung
 - OneDrive-Anbindung
 - Automatische Synchronisation
 - Fehlerbehandlung und Wiederholungsversuche
 - Offline-Pufferung für Upload-Warteschlange
- Automatisches Backup-System (Implementiert)
 - Konfigurierbare Backup-Intervalle (täglich/wöchentlich/monatlich)
 - Verschlüsselung der Backups mit AES-256
 - Intelligente Backup-Rotation mit Aufbewahrungsrichtlinien
 - Integritätsprüfung durch SHA-256 Checksummen
 - Automatische Bereinigung alter Backups
 - Wiederherstellungsprüfung
- Erweiterte Funktionen (Implementiert)
 - Datenkomprimierung mit Gzip
 - Teilen-Funktion für alle Exportformate
 - Ende-zu-Ende-Verschlüsselung
 - Umfassende Backup-Verwaltung
 - Detaillierte Export-Protokollierung
 - Batch-Export-Funktionalität

Services

ShoppingListService

Verwaltet Einkaufslisten und Produkte.

```
interface ShoppingListItem {
    id: string;
    name: string;
    quantity: number;
    unit: string;
    completed: boolean;
    category?: string | CustomCategory;
    addedAt: Date;
}
```

CategoryService

Verwaltung von benutzerdefinierten Kategorien.

```
interface CustomCategory {
  id: string;
  name: string;
  color: string;
  icon?: string;
  parentCategory?: string;
  subCategories?: string[];
  path: string[];           // Vollständiger Pfad zur Kategorie
  level: number;           // Hierarchieebene
  tags: string[];          // Kategorie-Tags für bessere Organisation
  metadata: {              // Erweiterte Metadaten
    description?: string;
    rules?: CategoryRule[];
    customFields?: Record<string, any>;
  };
  permissions: {           // Berechtigungssystem
    owner: string;
    sharedWith: string[];
    public: boolean;
    role: 'admin' | 'editor' | 'viewer';
  };
  status: 'active' | 'archived'; // Archivierungsstatus
  createdAt: string;
  lastModified: string;
  createdBy: string;
  modifiedBy: string;
}

interface CategoryRule {
  id: string;
  name: string;
  condition: {
    field: string;
    operator: 'contains' | 'equals' | 'startsWith' | 'endsWith' | 'regex';
    value: string;
  };
  priority: number;
  isActive: boolean;
  createdAt: string;
}
```

BudgetService

Tracking von Ausgaben und Budgets.

```
interface CategoryBudget {
  category: string;
  budget: number;
  period: 'daily' | 'weekly' | 'monthly';
  startDate: string;
}
```

NutritionService

Verwaltung von Ernährungsinformationen.

```
interface DailyNutrition {
  date: string;
  totalCalories: number;
  totalProteins: number;
  totalCarbohydrates: number;
  totalFat: number;
}
```

ExportService

Verwaltung von Exporten, Backups und Cloud-Integration.

```

interface ExportSettings {
    defaultFormat: ExportFormat;
    autoBackup: boolean;
    backupFrequency: 'daily' | 'weekly' | 'monthly';
    backupRetention: number;
    includeImages: boolean;
    encryptBackups: boolean;
    compressionEnabled: boolean;
    defaultCloudProvider?: CloudProvider;
}

interface ExportOptions {
    format: ExportFormat;
    cloudProvider?: CloudProvider;
    encrypt?: boolean;
    compress?: boolean;
}

type ExportFormat = 'json' | 'csv' | 'xml' | 'xlsx' | 'pdf';
type CloudProvider = 'google-drive' | 'dropbox' | 'onedrive';

interface ExportResult {
    success: boolean;
    format: ExportFormat;
    fileUri: string;
    fileName: string;
    size: number;
    cloudUploadStatus?: {
        uploaded: boolean;
        provider?: CloudProvider;
        path?: string;
        error?: string;
    };
}

interface BackupMetadata {
    id: string;
    createdAt: string;
    format: ExportFormat;
    size: number;
    checksum: string;
    cloudProvider?: CloudProvider;
    cloudPath?: string;
    compressed: boolean;
    contents: {
        shoppingLists: boolean;
        categories: boolean;
        budgets: boolean;
        nutrition: boolean;
        settings: boolean;
    };
}

```

RecipeService

Verwaltet Rezeptvorschläge und Zutatenlisten.

```

interface Recipe {
    id: string;
    name: string;
    ingredients: Ingredient[];
    instructions: string[];
    nutritionInfo: NutritionInfo;
}

```

FirebaseService

Zentraler Service für Firebase-Integration und Cloud-Funktionalitäten.

```

interface FirebaseConfig {
    apiKey: string;
    authDomain: string;
    projectId: string;
    storageBucket: string;
    messagingSenderId: string;
    appId: string;
}

class FirebaseService {
    // Authentication
    async signIn(email: string, password: string): Promise<UserCredential>;
    async signUp(email: string, password: string): Promise<UserCredential>;
    async signOut(): Promise<void>;

    // Firestore
    async syncData<T>(collection: string, localData: T[]): Promise<T[]>;
    async backupUserData(userId: string): Promise<void>;
}

```

```

// Storage
async uploadImage(uri: string, path: string): Promise<string>;
async downloadImage(path: string): Promise<string>;

// Cloud Functions
async processReceipt(imageUri: string): Promise<ReceiptData>;
async generateNutritionReport(): Promise<Report>;
}

## Komponenten

### CategoryForm
Formular für das Erstellen und Bearbeiten von Kategorien.
- Farbauswahl
- Icon-Auswahl
- Kategorieauswahl

### ProductCard
Wiederverwendbare Komponente für Produktanzeige.
- Produktinformationen
- Aktionsbuttons
- Kategoriefarben

### BudgetChart
Visualisierung von Budgetinformationen.
- Kreisdiagramm
- Fortschrittsbalken
- Farbkodierung

## Screens

### HomeScreen
Hauptbildschirm mit Übersicht über:
- Aktuelle Einkaufsliste
- Budget-Status
- Schnellzugriff auf Funktionen

### ShoppingListScreen
Verwaltung der Einkaufslisten:
- Liste der Produkte
- Sortierung nach Kategorien
- Scan-Funktion

### CategoryManagerScreen
Verwaltung der Kategorien:
- Liste aller Kategorien
- Bearbeiten/Löschen
- Neue Kategorie erstellen

### BudgetScreen
Budget-Übersicht und -Verwaltung:
- Aktuelle Ausgaben
- Budget-Einstellungen
- Warnungen

### NutritionScreen
Ernährungsinformationen:
- Tagesübersicht
- Zielvorgaben
- Trends

### StatisticsScreen
Auswertungen und Export:
- Grafische Darstellungen
- Exportoptionen
- Zeitraumauswahl

### RecipeSearchScreen
Rezeptvorschläge:
- Suche
- Filterung
- Detailansicht

## Datenmodelle

Die App verwendet verschiedene Datenmodelle für:
- Produkte
- Kategorien
- Budgets
- Ernährungsdaten
- Rezepte
- Statistiken

Alle Daten werden lokal mit AsyncStorage gespeichert.

```

```
## Konfiguration

### Firebase
Firebase-Konfiguration in `firebase.config.ts`:
```typescript
import { initializeApp } from 'firebase/app';

const firebaseConfig = {
 apiKey: 'your-api-key',
 authDomain: 'your-auth-domain',
 projectId: 'your-project-id',
 storageBucket: 'your-storage-bucket',
 messagingSenderId: 'your-sender-id',
 appId: 'your-app-id'
};

// Firebase initialisieren
const app = initializeApp(firebaseConfig);
export default app;
```

#### Firebase Security Rules:

```
rules_version = '2';
service cloud.firestore {
 match /databases/{database}/documents {
 // Benutzer können nur ihre eigenen Daten lesen und schreiben
 match /users/{userId}/{document=**} {
 allow read, write: if request.auth != null && request.auth.uid == userId;
 }

 // Öffentliche Kategorien können von allen gelesen werden
 match /categories/{category} {
 allow read: if true;
 allow write: if request.auth != null && request.auth.token.admin == true;
 }
 }
}
```

#### TypeScript

```
{
 "extends": "@tsconfig/react-native/tsconfig.json",
 "compilerOptions": {
 "strict": true,
 "lib": ["ES2019"],
 "types": ["react-native", "jest"]
 }
}
```

#### Expo

##### Verwendete Expo-Features:

- Camera
- FileSystem
- MediaLibrary
- Sharing
- AsyncStorage

## Installation und Setup

#### 1. Repository klonen:

```
git clone https://github.com/username/shoppingscanner.git
```

#### 2. Abhängigkeiten installieren:

```
cd shoppingscanner
npm install
```

#### 3. Firebase-Tools installieren und konfigurieren:

```
npm install -g firebase-tools
firebase login
firebase init
```

#### 4. Firebase-Konfiguration einrichten:

- Firebase Console öffnen
- Neues Projekt erstellen
- Web-App hinzufügen
- Konfigurationsdaten in `firebase.config.ts` einfügen

#### 5. Firebase-Abhängigkeiten installieren:

```
npm install @react-native-firebase/app @react-native-firebase/auth @react-native-firebase/firestore @react-native-firebase/storage
```

#### 6. Expo starten:

```
npx expo start
```

## Entwicklungsumgebung

- Node.js
- npm/yarn
- Expo CLI
- Android Studio / Xcode
- VS Code (empfohlen)

## VS Code Erweiterungen

- ESLint
- Prettier
- React Native Tools
- TypeScript and JavaScript Language Features

## Best Practices

### 1. Code-Struktur

- Komponenten-basierte Architektur
- Trennung von Logik und UI
- TypeScript für Typsicherheit

### 2. Firebase Best Practices

- Offline-First Ansatz mit Firestore
- Batch-Operations für mehrere Schreibvorgänge
- Security Rules testen
- Datenstruktur optimieren
- Authentifizierung und Autorisierung trennen
- Cache-Strategien implementieren

### 3. State Management

- React Hooks für lokalen State
- Context für globalen State
- AsyncStorage für Persistenz

### 4. Performance

- Memo für große Listen
- Lazy Loading
- Bildoptimierung

### 5. Testing

- Jest für Unit Tests
- React Native Testing Library
- E2E Tests mit Detox

## Implementierte Features

### 1. Smart Home Integration

- Anbindung an Smart-Kühlschränke (Implementiert)
  - Unterstützung für Samsung Family Hub
  - Unterstützung für LG ThinQ
  - Automatische Inventarerkennung
  - Echtzeit-Synchronisation
  - Temperaturüberwachung

### 2. Soziale Features

- Einkaufslisten teilen (Implementiert)
- Gemeinsame Haushaltsverwaltung (Implementiert)
  - Mehrbenutzer-Unterstützung
  - Rollenbasierte Zugriffsrechte
  - Echtzeit-Kollaboration

### 3. Datenschutz-Features (Implementiert)

- Ende-zu-Ende-Verschlüsselung für alle Daten
  - AES-256 Verschlüsselung
  - Sichere Schlüsselverwaltung
  - Verschlüsselte Datenübertragung
- Biometrische Authentifizierung
  - Fingerabdruck-Unterstützung
  - Face ID Integration
  - Mehrfaktor-Authentifizierung
- Erweiterte Datenschutzeinstellungen
  - Granulare Berechtigungskontrolle
  - Datenzugriffsprotokolle

- Automatische Datenlöschung
- DSGVO-Compliance-Tools
  - Datenexport-Funktion
  - Recht auf Vergessenwerden
  - Transparente Datenverarbeitung

#### 4. Erweiterte Soziale Features

- Rezeptaustausch
- Community-Preisvergleiche
- Sprachassistenten-Integration
- Automatische Nachbestellung
- IoT-Gerätesynchronisation (Implementiert)
  - MQTT-Client für IoT-Kommunikation
  - Geräteerkennung und automatische Verbindung
  - Echtzeit-Datensynchronisation
  - Fehlerbehandlung und Reconnect-Logik
  - Statusüberwachung und Benachrichtigungen